

Warehouse Inspector

Automated tile inspection with a U-Net

From a Colab notebook to the warehouse floor

Drop your job: you are inspectors now



As of today, you work at a recycling warehouse.

Whatever you were doing before, leave it. Your new job is **tile inspector**: look at every reclaimed tile and decide whether it can be **sold** or must be **recycled**.

The catch

Thousands of tiles arrive every day. You cannot check them all by eye, so you will build a model that does it for you.

What the Warehouse Inspector is

The problem

Reclaimed floor tiles arrive **by the thousand**, and today a person checks every one: *cracked? chipped? still sellable?*

 slow and tiring

 inspectors rarely agree on borderline tiles

What we built

An **inspection station**: a camera photographs each tile on the conveyor, a **U-Net** marks the usable surface, the station decides **sell or recycle**.

 people only see the tiles the model is unsure about



Photo

one tile on the belt



U-Net

belt, usable or damaged, per pixel



Usable share

$\text{usable} \div (\text{usable} + \text{damaged})$

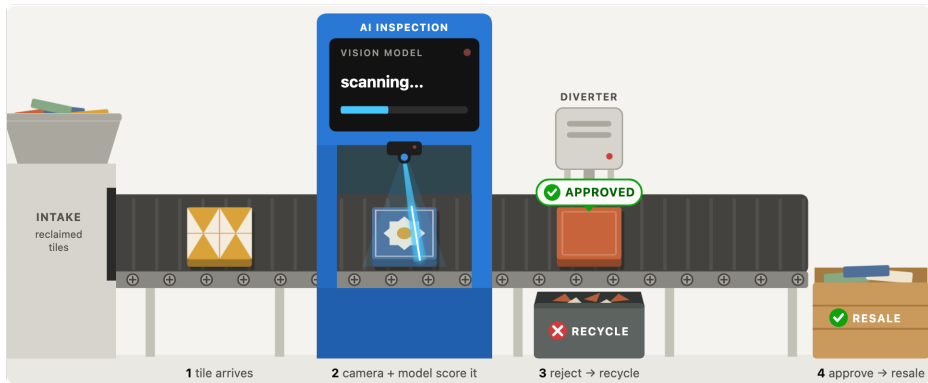


Verdict

APPROVE if $\geq 85\%$

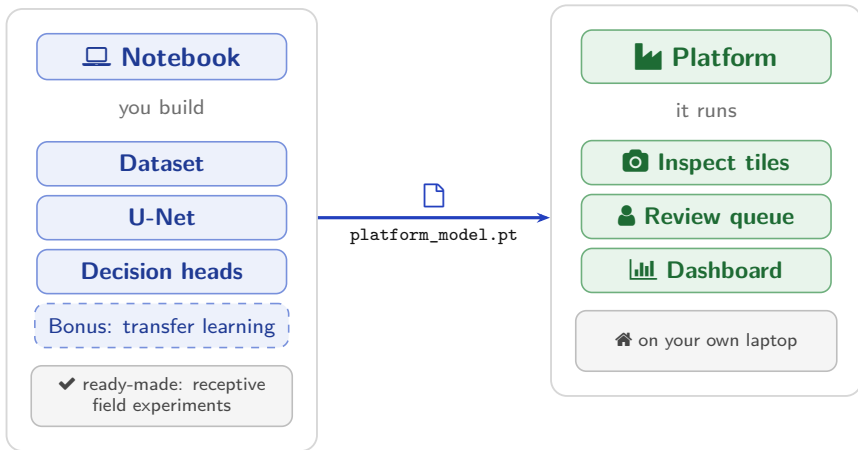
else **REJECT**

The inspection line



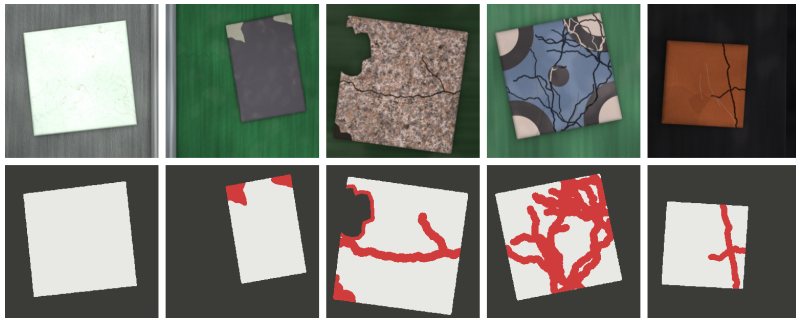
Tiles arrive from intake, the camera and the **vision model** score each one, and a diverter sends it to **resale** or **recycling**.

Two halves of the same thing



What the platform shows is exactly what you trained.

The data: one tile per photo



pristine marble

100 % ✓

5 000

photos with pixel-exact masks

chipped corners

92 % ✓

crack + chip

84.6 % ✗

55 %

approved; 37 % are pristine

crack network

54 % ✗

scratches are fine

89 % ✓

6 × 5

materials × belts, CC0 textures

Three classes, one rule

Every pixel is one of three classes

■	0 belt	not part of the tile
□	1 usable	surface that can be sold
■	2 damaged	cracks, chips, missing pieces + a 3% cutting margin

The hard part

Stains, dirt and scratches are **not** damage: the tile can be cleaned. A thin crack and a dark vein of marble look alike up close.

The decision rule

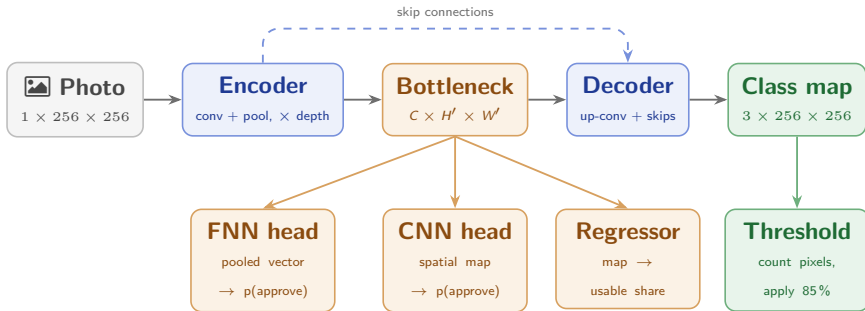
$$\text{usable share} = \frac{\text{usable pixels}}{\text{usable} + \text{damaged pixels}}$$

- $\geq 85\%$ → **APPROVE**: sell it, cut down if needed
- $< 85\%$ → **REJECT**: recycle it

Measured on the tile, not on the photo: a small tile far from the camera is judged like a big one.

One number, `data.USABLE_THRESHOLD`, used by the notebook and the platform.

The pipeline: four ways to reach a verdict



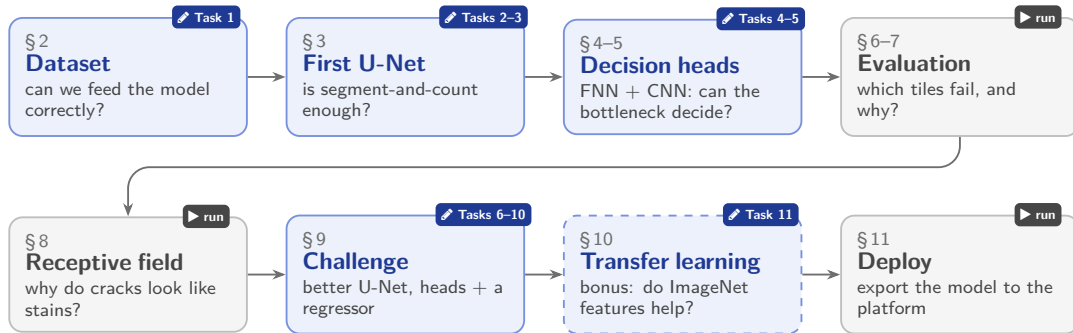
Segment, then count

The U-Net draws the map; the rule does the rest. Transparent: you can see why a tile was rejected.

Decide from the latent space

Small heads on the frozen bottleneck. What survives the compression, and does throwing away *where* hurt?

What students build, step by step

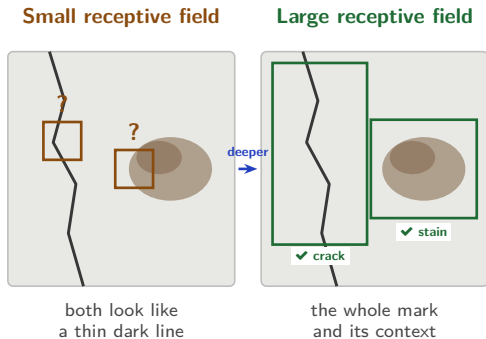


 you write code

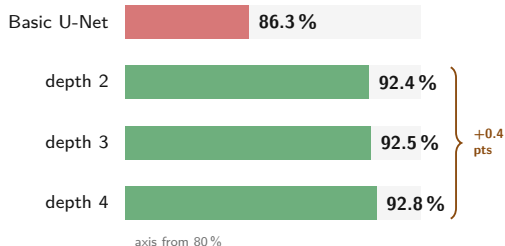
 ready-made: run and explore

🕒 every section ends with a *Why this step?* note

Why the first model fails: the receptive field



Threshold accuracy (test set)



💡 **Depth helps once, then saturates.** The rest needs a better model, better training and pretrained features.

§8, ready-made: train depth 2, 3, 4 ■ measure theoretical and effective receptive fields ■ click a tile to see what each model sees

The journey in numbers

How often is the **verdict right**? (U-Net + the 85 % rule, on 750 test tiles the models never saw)

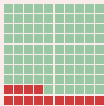
86.3 %

Basic U-Net

§3

≈ 103 of 750 wrong

✗ below the 96 % target



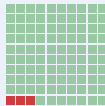
97.3 %

Improved U-Net

§9

≈ 20 of 750 wrong

✓ target ≥ 96 % met



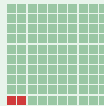
98.5 %

ResNet-18 U-Net

§10, pretrained

≈ 11 of 750 wrong

✓ target ≥ 96 % met



each square = 1 % of the test tiles ■ wrong verdict

Better masks

mean IoU 0.59 → 0.94 → 0.96
overlap with the true regions (1 = perfect)



Better usable share

error 7.5 → 1.9 → 0.9 points
fewer tiles pushed across the 85 % line

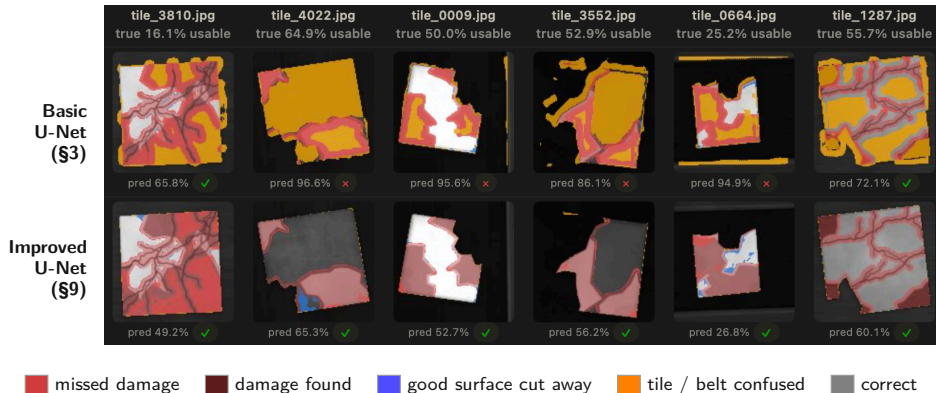


More correct verdicts

86.3 → 97.3 → 98.5 %
the big numbers above

💡 On the same improved U-Net, the heads decide less well than counting pixels: CNN **93.1 %**, regressor **91.7 %** vs. **97.3 %**.

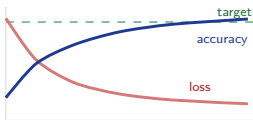
Same tiles, different models



The six test tiles where the basic model misses the most damage, from a real Colab run. The basic U-Net approves four of them; the improved one gets all six right.

An interactive notebook

Train



Live dashboard

curves against the target

Architecture diagram

every block with its shape

Evaluate



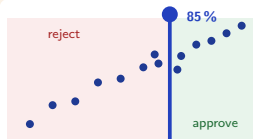
Scoreboard

all methods, all attempts

Confusion matrix + inspector

click a cell, see its error maps

Explore



Threshold & cost

drag the line, price errors

You vs. the model

a game: who inspects better?

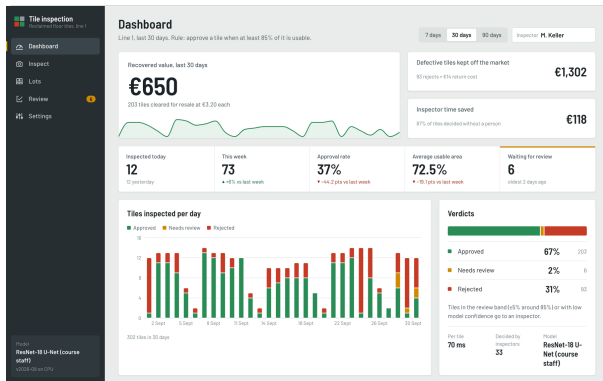
`</>` plain HTML, works in Colab ■  models saved to Drive ■ ► **RESUME HERE** in every section

The platform: what the warehouse runs

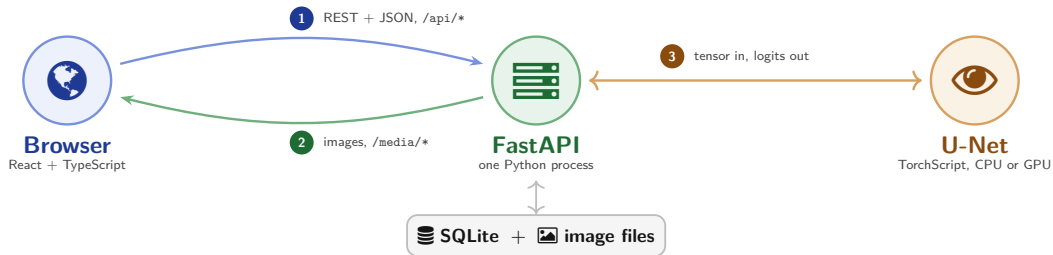
Dashboard	value recovered, approval rate, throughput, suppliers
Inspect	drop in photos or use the phone camera; watch the camera gate
Lots	one shipment: accept or reject, CSV, printable report
Review	borderline tiles for a person, least confident first
Settings	threshold, review band, prices, the model file

FastAPI + SQLite + React. No login; every decision is signed and audited.

Warehouse Inspector — automated tile inspection



How the pieces talk



1 A REST request

The browser sends photos and asks for data; JSON comes back.

```
POST /api/inspections
GET /api/review-queue
```

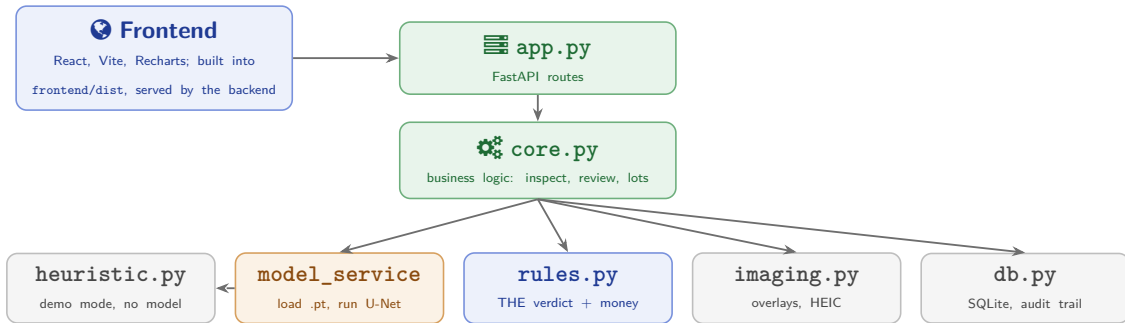
2 Stored images

Photos, overlays and masks are saved as files, so the database stays small and any inspection can be re-opened.

3 The model

Loaded once at start-up, with the notebook's preprocessing: grey, 256×256 , values in $[0, 1]$.

System architecture



One decision module. Nothing but `rules.py` decides a verdict, so the notebook's rule, the dashboard and the what-if in Settings can never disagree.

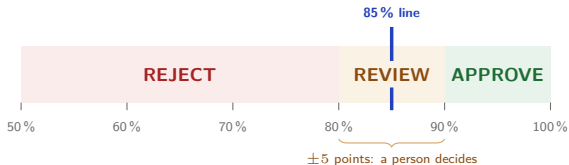
Every inspection stores the rule and the model version it was decided with; changing the rule never rewrites history.

One tile through the platform

⌚ ≈ 70 ms per tile on a laptop CPU (ResNet U-Net)



Decide: usable share



Confidence check

✓ far from 85 % × crisp mask

below 40 % → **REVIEW**,
whatever the share

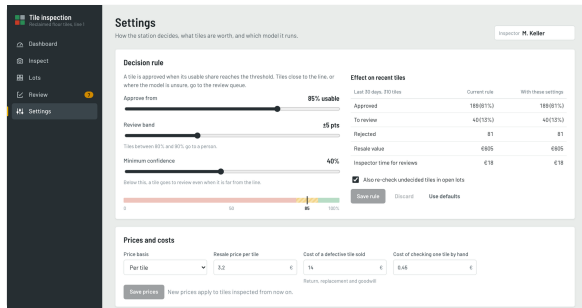
Threshold, band and minimum confidence are all adjustable in Settings.

From accuracy to money

Not every mistake costs the same

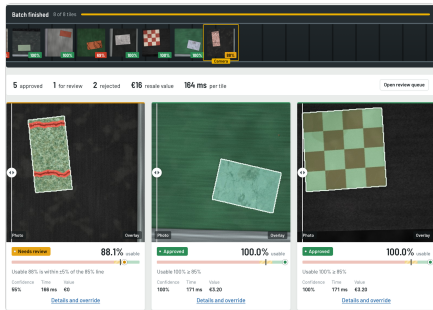
- a **false approval** ships a damaged tile: return, replacement, goodwill — **€ 14**
- a **false rejection** scraps a sellable tile — **€ 3.20**
- a **manual check** costs inspector time — **€ 0.45**

The notebook's cost explorer (§9.6) and the platform's what-if ask the same question: *where should the line and the review band sit?*



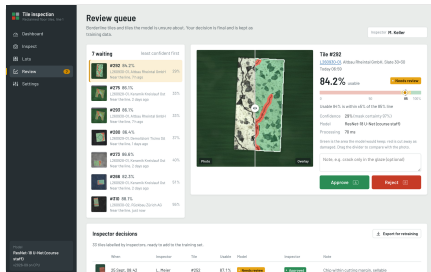
Settings: move a slider, see the last 30 days re-decided before saving.

What it looks like



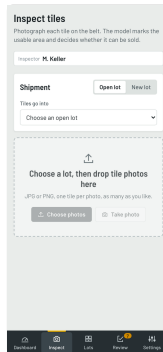
Inspect

a batch after the camera gate: overlay slider, share, verdict



Review

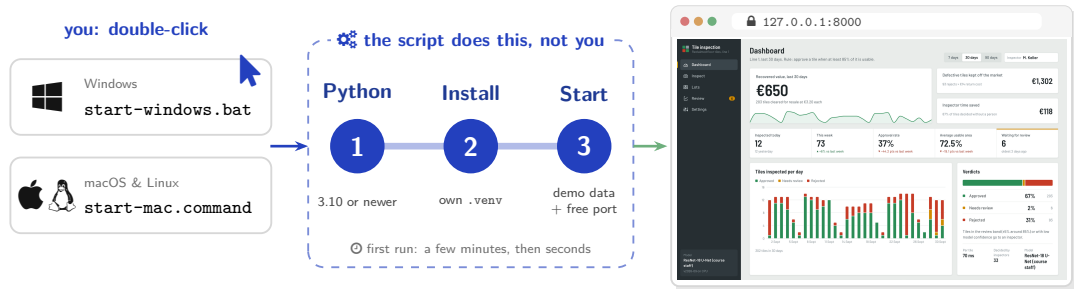
a person decides the borderline tiles; the decision becomes training data



Phone

camera capture on the same Wi-Fi

Getting it running: double-click

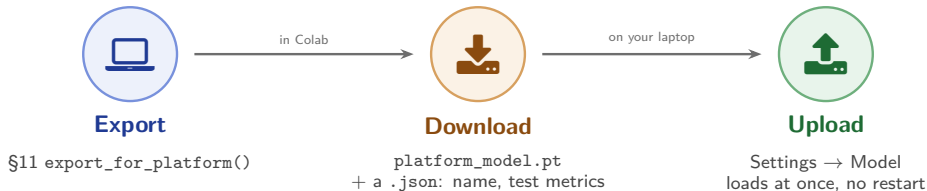


✓ No Node.js, nothing to build

🔒 Nothing installed system-wide

📦 A model included: works on day one

Your model in the platform



✓ For the platform: TorchScript

the **compiled network**: loads anywhere with PyTorch, no notebook code

✗ Not the notebook checkpoint

`save_model` (dill) needs the **same Python version**: fine for resuming, not for shipping

Any model works if it maps `(B,1,256,256) photo in [0,1]` → `(B,3,256,256) class logits`

In one picture



You were hired as inspectors.
Now you have **built one**.



Notebook

dataset, U-Net,
decision heads



Model file

platform_model.pt



Platform

inspect, review,
dashboard

A U-Net that **marks the usable surface**, decides **sell or recycle** and **hands the hard cases to a person**.

 `notebook.ipynb` in Colab

 `double-click start-... → localhost:8000`

Questions?

Warehouse Inspector — automated tile inspection, from notebook to warehouse floor
